MLVU information sheet

Please include this page in your report either at the start or at the end, before the appendix. Do not change the formatting.

Group number

156

Authors

name	student number
Conrad Clough	2699315
Samuel Power	2800107
ANDI FILL THIS OUT :)	THIS TOOOOO

Software used *Describe briefly which software and libraries you relied on. If you built a particular algorithm from scratch, make sure to state it explicitly here.*

Use of AI tools *If you used tools like ChatGPT or Github Copilot in any phase of the report, please detail here which tools you used, what parts of the work you used them for and how. Note that if you use an AI tool and don't report it, it's considered fraud, so be detailed. Note also that having an AI tool write for you is not allowed. If no AI tools were used, please state that explicitly.*

Link to code (optional) *If you would like to share your code, you can link to your repository here.*

Group disagreements (optional) *If there are any disagreements in the group, you may* ***not*** *remove a student from the author list without their consent. You should bring disagreements to our attention early. As a last resort, you can describe any grievances here.*

How can machine learning-based DDoS detection models be designed to minimise false positive rates while maintaining high detection accuracy on both low- and high-powered hardware?

Conrad Clough
VUNetId: cch250
Student Nr: 2699315
c.j.clough@student.vu.nl

Samuel Power
VUNetId: spo237
Student Nr: 2800107
s.j.power@student.vu.nl

mr. andy
VUNetId: x
Student Nr: x
lovelyman@student.vu.nl

I. Abstract

https://users.ece.cmu.edu/~koopman/essays/abstract.html
Checklist: Parts of an Abstract

Despite the fact that an abstract is quite brief, it must do almost as much work as the multi-page paper that follows it. In a computer architecture paper, this means that it should in most cases include the following sections. Each section is typically a single sentence, although there is room for creativity. In particular, the parts may be merged or spread among a set of sentences. Use the following as a checklist for your next abstract:

Motivation: Why do we care about the problem and the results? If the problem isn't obviously "interesting" it might be better to put motivation first; but if your work is incremental progress on a problem that is widely recognized as important, then it is probably better to put the problem statement first to indicate which piece of the larger problem you are breaking off to work on. This section should include the importance of your work, the difficulty of the area, and the impact it might have if successful. Problem statement: What problem are you trying to solve? What is the scope of your work (a generalized approach, or for a specific situation)? Be careful not to use too much jargon. In some cases it is appropriate to put the problem statement before the motivation, but usually this only works if most readers already understand why the problem is important. Approach: How did you go about solving or making progress on the problem? Did you use simulation, analytic models, prototype construction, or analysis of field data for an actual product? What was the extent of your work (did you look at one application program or a hundred programs in twenty different programming languages?) What important variables did you control, ignore, or measure? Results: What's the answer? Specifically, most good computer architecture papers conclude that something is so many percent faster, cheaper, smaller, or otherwise better than something else. Put the result there, in numbers. Avoid vague, hand-waving results such as "very", "small", or "significant." If you must be vague, you are only given license to do so when you can talk about orders-of-magnitude improvement. There is a tension here in that you should not provide numbers that can be easily misinterpreted, but on the other hand you don't have room for all the caveats. Conclusions: What are the implications of your answer? Is it going to change the world (unlikely), be a significant "win", be a nice hack, or simply serve as a road sign indicating that this path is a waste of time (all of the previous results are useful). Are your results general, potentially generalizable, or specific to a particular case?

Other Considerations

An abstract must be a fully self-contained, capsule description of the paper. It can't assume (or attempt to provoke) the reader into flipping through looking for an explanation of what is meant by some vague statement. It must make sense all by itself. Some points to consider include:

Meet the word count limitation. If your abstract runs too long, either it will be rejected or someone will take a chainsaw to it to get it down to size. Your purposes will be better served by doing the difficult task of cutting yourself, rather than leaving it to someone else who might be more interested in meeting size restrictions than in representing your efforts in the best possible manner. An abstract word limit of 150 to 200 words is common. Any major restrictions or limitations on the results should be stated, if only by using "weasel-words" such as "might", "could", "may", and "seem". Think of a half-dozen search phrases and keywords that people looking for your work might use. Be sure that those exact phrases appear in your abstract, so that they will turn up at the top of a search result listing. Usually the context of a paper is set by the publication it appears in (for example, IEEE Computer magazine's articles are generally about computer technology). But, if your paper appears in a somewhat un-traditional venue, be sure to include in the problem statement the domain or topic area that it is really applicable to. Some publications request "keywords". These have two purposes. They are used to facilitate keyword index searches, which are greatly reduced in importance now that on-line abstract text searching is commonly used. However, they are also used to assign papers to review committees or editors, which can be extremely important to your fate. So make sure that the keywords you pick make assigning your paper to a review category obvious (for example, if there is a list of conference topics, use your chosen topic area as one of the keyword tuples).

Distribution Denial of Service (DDoS) attacks continue to disrupt network services, causing downtime and economic damage. Existing techniques for mitigating this often struggle to whatever they struggle with, find a paper and cite it. This paper investigates the design of DDoS detection models using machine learning. We evaluate three classifiers, k-Nearest Neighbours (kNN), Logistic Regression (LR), and Random Forests, by analysing their accuracy, false positive rates, and resource usage under controlled conditions. Our results indicate that whatever we find. One sentence about each, what they do well and what they do bad. These findings suggest that model selection is dependent on hardware specifications and the relative importance of minimising false positives, providing a practical guide for using machine learning-based DDoS detection in diverse contexts.

Keywords: DDoS detection, machine learning, false positive rate, k-Nearest Neighbours, Random Forest, Logistic Regression, computational efficiency

II. Introduction

The proliferation of online services has made networks increasingly vulnerable to Distributed Denial of Service (DDoS) attacks, which can overwhelm servers and disrupt them, causing significant operational downtime and economic damage. Detecting these attacks efficiently is challenging because high accuracy must be balanced with low false positive rates; denying service to legitimate users is highly undesirable, as is allowing too much malicious traffic through. Machine learning offers a promising alternative to traditional, rule-based approaches; patterns in network traffic can be used to identify evolving malicious activity in

real time. This document investigates the design of machine learning-based DDoS detection models with an emphasis on maintaining high detection accuracy while reducing false positives, and evaluates their resource usage across a range of hardware to ensure usability for a variety of situations. Specifically, k-Nearest Neighbours (kNN), Logistic Regression (LR), and Random Forest classifiers are compared in terms of both accuracy, false positive rate, and performance.

Blah blah blah. We can quote code like this. We can also reference sections like this Section II or even define labels on things like figures and images and reference them later.

Wanna reference something in the bibliography? Just do it with another label [1]

The report is structured as follows:

- In Section III we consider and justify our design.

III. Design

The algorithms in this project were trained on data obtained from kaggle (MAYBE NEED TO LINK HERE??). The data was already mostly balanced between benign traffic and DDoS traffic. The data was also shuffled in the event that some unknown order had been preserved upon creation. Notably though the data is quite large and does not fit into the memory of the hardware of the researchers. For this reason, the project employs a chunking approach, so that the data can be trained upon. The data is read into the models in chunks. Additionally, the entire dataset is not loaded in as it is first perfectly balanced by the researchers. This is done to ensure that the dataset remains balanced, which is important to prevent models from favoring the majority class. This ensures accurate, fair, and reliable predictions across all classes. In this case, 10'000 rows for DDoS and 10'000 rows for benign traffic were used. The features were also modified. the identifying columns and other non-revelvant determined data were all dropped. The dropper columns were as follows: "Label", "Unnamed: 0", "Flow ID", "Src IP", "Dst IP", "Timestamp", "Src Port", "Dst Port", were all dropped [NEED TO FORMAT THIS STILL]

The protocol was additionally one-hot encoded since it is categorical. The data was also cleaned, where inf values were relaced with NaN. For robustness, outliers were identified through use of the median instead of the mean. In terms of training the models a 60/20/20 split was employed where 60% of the data was used for training, 60% for validation and 20% for testing. Add still: how is correctness evaluated? feature scaling in KNN and LR (not present in RF), hyper param tuning, and maybe eval/feature importances

[W I L L | F I N I S H | A N D | F O R M A T | S O O N -SAMUEL]

- Bullet points are super easy
- Like this!

1) Numbered lists too
2) Wow
    a) Even with indentation
    - and mixing!

Math, with equation labels:

$$T_{\text{coarse}} = ctf(n) \tag{1}$$

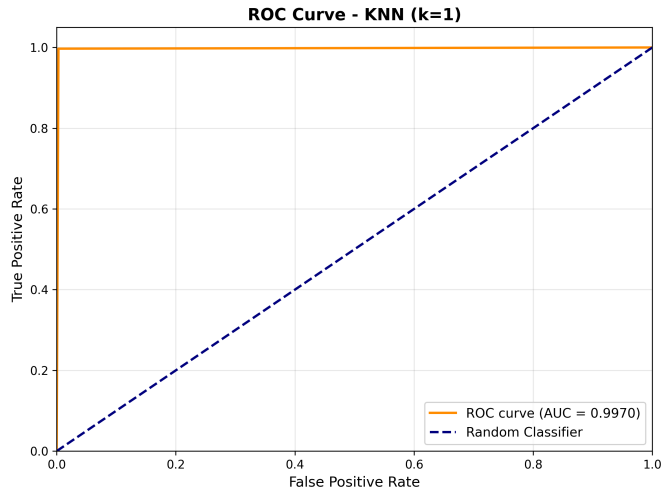

Fig. 1. It's good practice to put figures at the top of the page and just cite them via label

TABLE I
Amdahl's Law metrics at 65536 job count

Structure	p	Max Speedup	Eff. (16 threads)
Coarse-grained List	2.3%	1.02×	6.4%
Coarse-grained BST	92.7%	13.64×	45.4%
Fine-grained List	48.5%	1.94×	22.9%
Fine-grained BST	76.2%	4.20×	19.3%

. Or even inline $\frac{n}{2}$ wowie

$$T_{\text{fine}} = \max\left(\left\lceil \frac{c}{2n} \right\rceil \cdot tf(n), tf(n)\right) \tag{2}$$

.

Figures! They CANNOT be in a super directory, like ../plots, so we have to use ./images/

In Fig. 1 we can very clearly see that our experimental data matches the time complexity of access/mutation for our structures.

Tables too:

Generally you can surround things in figures or blocks and such, it's all super functional, the typst website will guide you

Typst website also have all the symbol names, and some symbols have subsets like $\subset$ has $\subseteq$

References

[1] Oracle, "Java Class ReentrantLock." [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/locks/ReentrantLock.html